

① $\rightarrow$ The worst-case running time of quicksort is $\Theta(n^2)$. It is often the best practical choice for sorting because its average running time is $\Theta(n \log n)$ and the constant factors in the $\Theta(n \log n)$ notation are quite small.

$\rightarrow$ Quicksort is based on the divide-and-conquer paradigm.

The process for sorting a subarray $A[p..r]$ is as follows:-

- ① Divide: Partition the array $A[p..r]$ into two subarrays $A[p..q-1]$ and $A[q+1..r]$ such that each element of $A[p..q-1]$ is less than or equal to $A[q]$, while each element of $A[q+1..r]$ is greater than or equal to $A[q]$. Compute the index q as a part of this partitioning procedure.
- ② Conquer: Sort the two subarrays $A[p..q-1]$ and $A[q+1..r]$ by recursive calls to quicksort.
- ③ Combine: Since the subarrays are sorted in place, no effort is needed to combine them.

$\rightarrow$ QUICKSORT(A, p, r)

- 1 if $p < r$
- 2 then $q \leftarrow \text{PARTITION}(A, p, r)$
- 3 QUICKSORT($A, p, q-1$)
- 4 QUICKSORT($A, q+1, r$)

$\triangleright$ To sort an entire array A , the initial call is
QUICKSORT($A, 1, \text{length}[A]$)

② Partitioning the array

The PARTITION procedure selects an element $x = A[r]$ as a pivot element around which to partition the subarray $A[p..r]$.

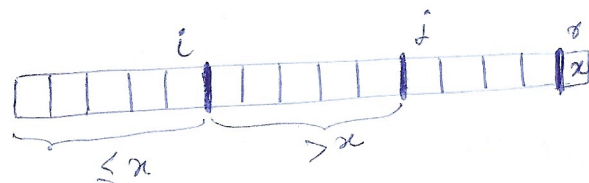
As the procedure runs, the array is partitioned into (2) 4 sub-regions: (i) the region bound by i with values no greater than x (ii) the region bound by j with values no less than x (iii) the untouched elements of the array (iv) $x = A[p] = \text{pivot element}$.

PARTITION (A, p, r)

```

1  $x \leftarrow A[p]$ 
2  $i \leftarrow p-1$ 
3 for  $j \leftarrow p$  to  $r-1$ 
4   do if  $A[j] \leq x$ 
5       then  $i \leftarrow i+1$ 
6           exchange  $A[i] \leftrightarrow A[j]$ 
7 exchange  $A[i+1] \leftrightarrow x$ 
8 return  $i+1$ 

```



→ The running time of PARTITION is $\Theta(n)$ where $n = r - p + 1$

③ Performance of quicksort

The running time of quicksort depends on whether the partitioning is balanced or unbalanced.

→ Worst-case partitioning: occurs when the partitioning routine produces one subproblem with $n-1$ elements and one with 0 elements.

→ partitioning cost

$$T(n) = T(n-1) + T(0) + \Theta(n)$$

$$\hookrightarrow \Theta(1)$$

$$= T(n-1) + \Theta(n)$$

The solution to this recurrence is $\Theta(n^2)$.

→ Best case partitioning, each partition is of size $n/2$ approx. (3)

$$T(n) \leq 2T(n/2) + \Theta(n)$$

The solution to this recurrence is $T(n) = O(n \lg n)$.

Balanced partitioning

Suppose, that the partitioning algorithm always produces a q -to- 1 split,

$$T(n) \leq T\left(\frac{q}{10}n\right) + T\left(\frac{9}{10}n\right) + cn$$

↓
↳ This recursion will terminate at depth $\log_{10} n$

This recursion will terminate at depth $\log_{10} n$

At every step the cost is $\Theta(n)$, and there are $O(\lg n)$ such steps. Hence, quicksort is $O(n \lg n)$.

In the average case PARTITION produces a mix of good and bad splits and its running time is $O(n \lg n)$, (with a slightly larger constant than the best case).

④ A randomized version of quicksort

Instead of always using $A[0]$ as the pivot, we randomly choose an element from the subarray $A[p..r]$ and exchange it with $A[r]$.

RANDOMIZED-PARTITION(A, p, r)

- 1 $i \leftarrow \text{RANDOM}(p, r)$
- 2 exchange $A[0] \leftrightarrow A[i]$
- 3 return PARTITION(A, p, r)

RANDOMIZED-QUICKSORT(A, p, r)

- 1 if $p < r$
- 2 then $q \leftarrow \text{RANDOMIZED-PARTITION}(A, p, r)$
- 3 RANDOMIZED-QUICKSORT($A, p, q-1$)
- 4 RANDOMIZED-QUICKSORT($A, q+1, r$)

_____ x x x _____